Milestone 3 — Chunked Prefill + Radix Prefix Cache

Author: Joey Ji (junyiji3@stanford.edu) | Thomason Zhao (changyuz@stanford.edu)

All experiments below ran on a single NVIDIA L4 serving `Qwen/Qwen3-8B` through MiniEngine's paged-mode engine with the FlashInfer attention backend.

1. Design and implementation

This milestone layers two memory-pressure controls on top of the milestone-2 paged engine: **chunked prefill** to bound activation memory during long-prompt prefills, and a **radix prefix cache** to skip prefill entirely on shared-prefix workloads. A third, bonus component — **decode-time retraction** — recovers from KV-pool exhaustion that can still occur during generation. The three pieces interact closely: chunking changes what the scheduler does *during* prefill, the cache changes what the scheduler does *before* prefill, and retraction is the safety net that catches the rare case neither prevents.

1.1 Chunked prefill

The milestone-2 prefill step processes every admitted request's entire prompt in a single packed varlen forward. Because the MLP intermediate and attention activations are sized by the total number of q-tokens in that forward, a high-concurrency batch of long prompts can transiently exceed the GPU's free activation memory and OOM the engine. The most important observation here is that this is *not* a steady-state memory problem — the activation tensors are short-lived per layer — but a peak-allocation problem driven by the single forward's q-token total.

The design splits each admitted request's prompt into fixed-size chunks and advances every active prefill by exactly one chunk per scheduler step. Admission still reserves the full prompt's KV pages up front (the cache integration depends on knowing the full prompt's page count), but the *forward* is now bounded: per step, the activation footprint is governed by `chunk_size × |active prefills|`, not by the per-request prompt length. The scheduler keeps a separate `prefilling` list of admitted-but-not-yet-finished-prefill requests; a request graduates into the decoding running set only when its final chunk produces a first sampled token.

Concretely, the chunked-prefill forward packs all currently-prefilling requests' next chunks into one varlen call. Each request contributes a token range `[start, end)`, where `start = prefill_offset` and `end = min(start + chunk_size, prefill_target)`. The attention backend (FlashInfer) is told the per-request *cumulative* KV length so it can reuse already-written K/V pages from earlier chunks. Importantly, the engine only samples a first token for requests whose `end == prefill_target` in this step; partial-chunk completions return `None` and stay in the prefilling list for the next step. This produces the deterministic phase order — admit, prefill-one-chunk-for-all, decode-running-set — that keeps the implementation tractable and predictable.

The cost of chunking is one extra attention-plan and kernel launch per chunk per step. We characterize this cost in §2.6: at low concurrency it's dwarfed by per-chunk compute, at high concurrency it's small relative to the head-of-line block, but at moderate concurrency it can become visible.

1.2 Radix prefix cache

The cache is a trie keyed by token pages: each tree node owns a contiguous run of physical KV pages from the pool, and an incoming request's prompt walks the trie one page at a time to find the longest already-computed prefix. The critical design decision is **page granularity**: a hit is always an integer number of pages, never a partial page. This is what makes the cache compatible with the engine's attention metadata — the page table and the last-page slot mapping are page-aligned by construction, so a page-aligned hit can be stitched directly into a request's `page_table` without any per-token surgery. The cost is that a 2000-token shared prefix matches as 62 pages × 32 = 1984 tokens; the trailing 16 tokens get re-prefilled. We accept that loss because page-aligned matching is simpler, faster, and provably consistent with the rest of the paged engine.

Three invariants drive the rest of the design. **First**, the cache and the pool's free list must never disagree about which pages are free. We enforce this by making the cache *the* eviction hook: when the pool runs out of free pages, it consults the cache's `evict` callback before raising OOM. The cache then walks its leaves in oldest-access-first order, frees pages whose nodes have a zero lock count, and returns those pages to the pool. There is no separate "garbage collection" step — eviction happens on demand, lazily, exactly when the pool needs the pages.

Second, an in-flight request whose cached pages we just lent out must not have those pages evicted from under it. We implement this with reference-counted lock pinning: at admission, we increment the lock count of the matched node and every ancestor up to the root; when the request retires, we decrement. The eviction walk skips any node with a positive lock count. This is essentially SGLang's design — leaves are the only evictable nodes, and locking a deep node automatically pins its entire path.

Third, a request must always have at least one prompt token left to actually prefill, because the cache stores K/V but not the logits we need to sample the first generated token. The scheduler enforces this by clamping the match length to `(prefill_len - 1) // page_size * page_size` before consulting the cache, so the suffix is always at least one page short of the full prompt.

One policy choice worth calling out: we cache against `prompt + generation`, not just the prompt. When a request retires, the page-aligned concatenation of its input and output IDs is inserted into the trie. The direct consequence is that a multi-turn conversation, where each turn embeds the prior assistant response, hits the cache on that prior response — not just on the user-resent system prompt. This is the mechanism behind the monotonic per-turn hit-rate climb shown in §2.3 (deliverable 6).

1.3 Decode-time retraction

Chunking and the cache together largely solve prefill-side memory pressure, but they cannot prevent decode-time OOM. A request's generation length is unknown at admission; if a sufficiently large running set keeps growing tokens without retiring, the pool eventually drains and the next per-request page allocation during decode fails. Without recovery, the scheduler thread crashes with an uncaught `RuntimeError`.

The design choice was to raise a *typed* exception — `PagedDecodeOOM` — at the exact site where decode-time page growth fails, after the pool has already given the cache a chance to evict unlocked pages. The scheduler catches this exception, picks a victim from the active set, frees the victim's pool-owned KV pages back to the pool, and pushes it back onto the waiting queue for re-admission and replay-prefill. The retry loop is bounded by the size of the active set, so forward progress is guaranteed.

Victim selection is a deliberate, ordered heuristic rather than a simple LRU. The ordering is: first prefer the request whose own allocation triggered the OOM (it is the one currently causing pressure); among the rest, free the most pages possible (largest page table); ties go to the request with the largest *remaining* output budget (it has the most future growth still to do); the final tie-break is *youngest* arrival time, which intentionally protects older FCFS-fair requests from avoidable retraction. This ordering optimizes for the most pressure relieved per retraction event while keeping the queueing discipline predictable.

Two subtler cleanup details matter. First, a retracted request's borrowed cache pages are *unlocked* but not freed — we want the next admission to be able to re-hit the same prefix. Only the pool-owned (non-cache) pages return to the free list. Second, when the retracted request is re-admitted, the replay prefill processes `input_ids + output_ids_so_far`, not just `input_ids`. This restores the client-visible prefix without re-streaming tokens to the client (the engine had already emitted them) and lets the cache provide hits on the partial generation if a sibling request had already populated the same prefix.

2. Performance

2.1 Benchmarking setup

Item	Value
GPU	NVIDIA L4 (24 GiB HBM)
Model	`Qwen/Qwen3-8B` (fp16, 8.19 B params)
Engine	MiniEngine paged mode, `--page-size 32 --max-running 16`, FlashInfer attention backend
Software	torch 2.5.1+cu121 · flashinfer-python 0.2.5+cu121torch2.5 · cuda-toolkit 12.6 (system)
KV pool budget	`--mem-fraction-static 0.85` → 610 pages × 32 tokens = ~19,520 cacheable tokens (except deliverables 1 and 8/9 which set this deliberately)
Chunk size (Part A)	`--prefill-chunk-size 1024` (chosen below in §2.6)
Cache (Part B)	enabled by default; turned off with `--disable-radix-cache` for off-runs

2.2 Part A — Chunked prefill

Deliverable 1 — OOM avoidance demo

Identical workload, two server configs at `--mem-fraction-static 0.96` (this tightens the activation budget to ~1 GiB so a moderate packed prefill exceeds it):

`bench_serving --input-len 4096 --output-len 16 --concurrencies 8 --num-requests 8`

Config	Server-side outcome
`--prefill-chunk-size 0` (unchunked)	`torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 450.00 MiB. ... 21.73 GiB memory in use.`
`--prefill-chunk-size 1024` (chunked)	clean shutdown; no OOM

```
----- UNCHUNKED (--prefill-chunk-size 0): expected to OOM -----

[server log excerpt - CUDA OOM]
                        ^^^^^^^^
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 450.00 MiB. GPU 0 has a total capacity of 22.04 GiB of which 291.88 MiB is free. Process 2367624
 has 21.73 GiB memory in use. Of the allocated memory 21.26 GiB is allocated by PyTorch, and 254.35 MiB is reserved by PyTorch but unallocated. If reserved bu
t unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See documentation for Memory Management  (
https://pytorch.org/docs/stable/notes/cuda.html#environment-variables)
20:29:37 INFO     miniengine.scheduler  Finished request 55a1c57d-4c2c-4e29-8aa2-18fffcd770ab  (output_len=16, running=1, waiting=0)

[bench client output]
  Loading tokenizer...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
  Loading WildChat prompts...
  Loaded 40 candidate prompts
  Preparing 8 requests...
  Input lengths:  min=4098, max=4105, mean=4102
  Output lengths: min=16, max=16, mean=16

  Running concurrency=8 (8 requests, closed-loop)...
    [  1/8] ok in=4105 out=16 ttft=22150ms compl=33143ms (elapsed 33.1s)
```

```
----- CHUNKED (--prefill-chunk-size 1024): completes successfully -----

[bench client output - all 8 requests OK]
  Loading tokenizer...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
  Loading WildChat prompts...
  Loaded 40 candidate prompts
  Preparing 8 requests...
  Input lengths:  min=4098, max=4105, mean=4102
  Output lengths: min=16, max=16, mean=16

  Running concurrency=8 (8 requests, closed-loop)...
    [  1/8] ok in=4105 out=16 ttft=11665ms compl=13107ms (elapsed 13.1s)
    [  2/8] ok in=4098 out=16 ttft=11864ms compl=13193ms (elapsed 13.2s)
    [  3/8] ok in=4105 out=16 ttft=11864ms compl=13194ms (elapsed 13.2s)
    [  4/8] ok in=4098 out=16 ttft=11864ms compl=13194ms (elapsed 13.2s)
    [  5/8] ok in=4105 out=16 ttft=11864ms compl=13194ms (elapsed 13.2s)
    [  6/8] ok in=4098 out=16 ttft=11864ms compl=13194ms (elapsed 13.2s)
    [  7/8] ok in=4105 out=16 ttft=11863ms compl=13194ms (elapsed 13.2s)
    [  8/8] ok in=4105 out=16 ttft=11864ms compl=13194ms (elapsed 13.2s)
  conc=8      TTFT p50=  11864ms  p99=  11864ms  Compl p50=  13194ms  p99=  13194ms  GenTok/s=     10  ok=8

=============================================================================================================
  Conc   TTFT_p50   TTFT_p99  Compl_p50  Compl_p99   TPOT_p50   TPOT_p99   GenTok/s    OK
       (ms)       (ms)       (ms)       (ms)       (ms)       (ms)
-------------------------------------------------------------------------------------------------------------
     8      11864      11864      13194      13194       88.7       95.6         10     8
=============================================================================================================
```

The packed prefill in the unchunked run tries to materialize MLP gate/up activations for 8 × 4096 ≈ 32K q-tokens at once — ~32K × 12,288 × 2 B ≈ 770 MiB *per layer*; with 36 layers, transient peaks blow past the 1 GiB budget. With chunk=1024, each step processes 8 × 1024 = 8K q-tokens (~200 MiB MLP), which fits comfortably.

Deliverable 2 — MMLU accuracy (lossless)

```
bench_accuracy --dataset mmlu --num-samples 200 --concurrency 8
```

Server flags: `--mem-fraction-static 0.85`, `--prefill-chunk-size 1024` (or `0` for unchunked).

Chunk size	Samples	Correct	Accuracy	Avg latency
1024 (chunked)	200	123	**61.5 %**	1.52 s
0 (unchunked)	200	123	**61.5 %**	1.51 s

Identical accuracy and identical incorrect-prediction set on the first five inspected samples. Chunked prefill is numerically lossless.

```
----- CHUNKED (--prefill-chunk-size 1024) -----

  MMLU Accuracy Report
  Samples    : 200
  Correct    : 123
  Accuracy   : 61.5%
  Avg latency: 1.52s
============================================================

----- UNCHUNKED (--prefill-chunk-size 0) -----

  MMLU Accuracy Report
  Samples    : 200
  Correct    : 123
  Accuracy   : 61.5%
  Avg latency: 1.51s
============================================================

----- VERDICT: identical 61.5% accuracy on both → lossless ✓ -----
```

Deliverable 3 — `bench_serving` regression conc 1 / 4 / 16

```
bench_serving --input-len 4096 --output-len 512 --concurrencies 1,4,16
```

chunked at `--prefill-chunk-size 1024` (≈ 4 chunks per request).

Conc	Chunked TTFT p50 / p99	Unchunked TTFT p50 / p99	Δ TTFT p50	TPOT (chunked / unchunked)	GenTok/s
1	1408 / 1607 ms	1498 / 1673 ms	**–6 %**	~70 ms each	14 / 14
4	5836 / 5882 ms	4432 / 5902 ms	**+31 %**	~83 ms each	44 / 44
16	143412 / 144272 ms	140909 / 143864 ms	**+2 %**	~85 ms each	44 / 45

TPOT and throughput differ by < 2 % at all three concurrencies. TTFT is within noise at conc=1 and conc=16, but shows a real +31 % regression at conc=4.

The conc=4 outlier is **per-chunk overhead at moderate concurrency**: each step calls `flashinfer.BatchPrefillWithPagedKVCacheWrapper.plan` and a varlen forward; the plan and kernel-launch cost is ~hundreds of milliseconds and amortizes over the work in the step. At conc=1 the chunked path has very little parallel work so the overhead is dwarfed by the per-chunk compute; at conc=16 the head-of-line block dominates and per-step overhead is again small relative to compute. At conc=4 the activation batch is exactly the "medium" size where adding 3 extra plan-and-launch cycles is most visible.

```
===== Milestone 3 Deliverable 03: bench_serving chunked vs unchunked, conc 1/4/16 =====
Workload: bench_serving --input-len 4096 --output-len 512 --concurrencies 1,4,16
Server flags: --mem-fraction-static 0.85

----- CHUNKED (--prefill-chunk-size 1024) -----
============================================================
============================================================
  conc=1      TTFT p50=   1408ms  p99=   1607ms  Compl p50=  35704ms  p99=  35847ms  GenTok/s=     14  ok=8
  conc=4      TTFT p50=   5836ms  p99=   5882ms  Compl p50=  46048ms  p99=  46081ms  GenTok/s=     44  ok=8
  conc=16     TTFT p50= 143412ms  p99= 144272ms  Compl p50= 184219ms  p99= 185451ms  GenTok/s=     44  ok=32
================================================================================================================
================================================================================================================

----- UNCHUNKED (--prefill-chunk-size 0) -----
============================================================
============================================================
  conc=1      TTFT p50=   1498ms  p99=   1673ms  Compl p50=  35784ms  p99=  35903ms  GenTok/s=     14  ok=8
  conc=4      TTFT p50=   4432ms  p99=   5902ms  Compl p50=  46102ms  p99=  46105ms  GenTok/s=     44  ok=8
  conc=16     TTFT p50= 140909ms  p99= 143864ms  Compl p50= 184004ms  p99= 186978ms  GenTok/s=     45  ok=32
================================================================================================================
================================================================================================================
```

2.3 Part B — Radix prefix cache

Deliverable 4 — Shared workload (≥ 2× targets)

```
bench_cache --workload shared --num-groups 5 --questions-per-group 20 \
            --shared-prefix-len 2000 --concurrency 4
```

Cache	Wall time	Throughput	TTFT p50 / p99	Hit rate
On	47.57 s	**2.10 req/s**	**232 / 3455 ms**	78.9 %
Off	108.81 s	0.92 req/s	3232 / 3442 ms	0.0 %

- Throughput: **2.10 / 0.92 = 2.28× ✓ (target ≥ 2×)**
- TTFT p50: **3232 / 232 = 13.9× reduction ✓ (target ≥ 2×)**

`/cache_stats` after the run: `hit_rate=0.789, total_inserted_pages=460, total_evicted_pages=92, num_cached_pages=368` . The cache holds 5 groups × ~62 prefix pages ≈ 310 locked pages of shared prefix; the 92 evictions are old suffixes being recycled from the tail. The 78.9 % hit rate is right at the page-aligned theoretical maximum (a 2000-token prefix matches as 62 pages × 32 = 1984 tokens; the per-question suffix never hits).

```
----- CACHE ON -----

  Wall time         : 47.57 s
  Throughput        :   2.10 req/s
  Gen tok/s         :     34
  TTFT p50 / p99    :   232 ms /  3455 ms
  Latency p50 / p99 :  1283 ms /  4566 ms
  Prompt tokens     : 233690
  Cache-hit tokens  : 184320
  Cache hit rate    : 78.9%
============================================================

----- CACHE OFF (--disable-radix-cache) -----

  Wall time         : 108.81 s
  Throughput        :   0.92 req/s
  Gen tok/s         :     15
  TTFT p50 / p99    :  3232 ms /  3442 ms
  Latency p50 / p99 :  4345 ms /  4558 ms
  Prompt tokens     : 233690
  Cache-hit tokens  : 0
  Cache hit rate    : 0.0%
============================================================

----- VERDICT -----
Throughput speedup: 2.10 / 0.92 = 2.28x  (target ≥ 2x) ✓ PASS
TTFT reduction:    3232 / 232 = 13.9x   (target ≥ 2x) ✓ PASS

{"enabled":true,"hit_rate":0.7887372159698747,"total_lookups":100,"total_query_tokens":233690,"total_hit_tokens":184320,"total_inserted_pages":460,"total_evic
ted_pages":92,"num_cached_pages":368,"num_evictable_pages":368,"pool_num_free":241,"pool_num_evictable":368}%
```

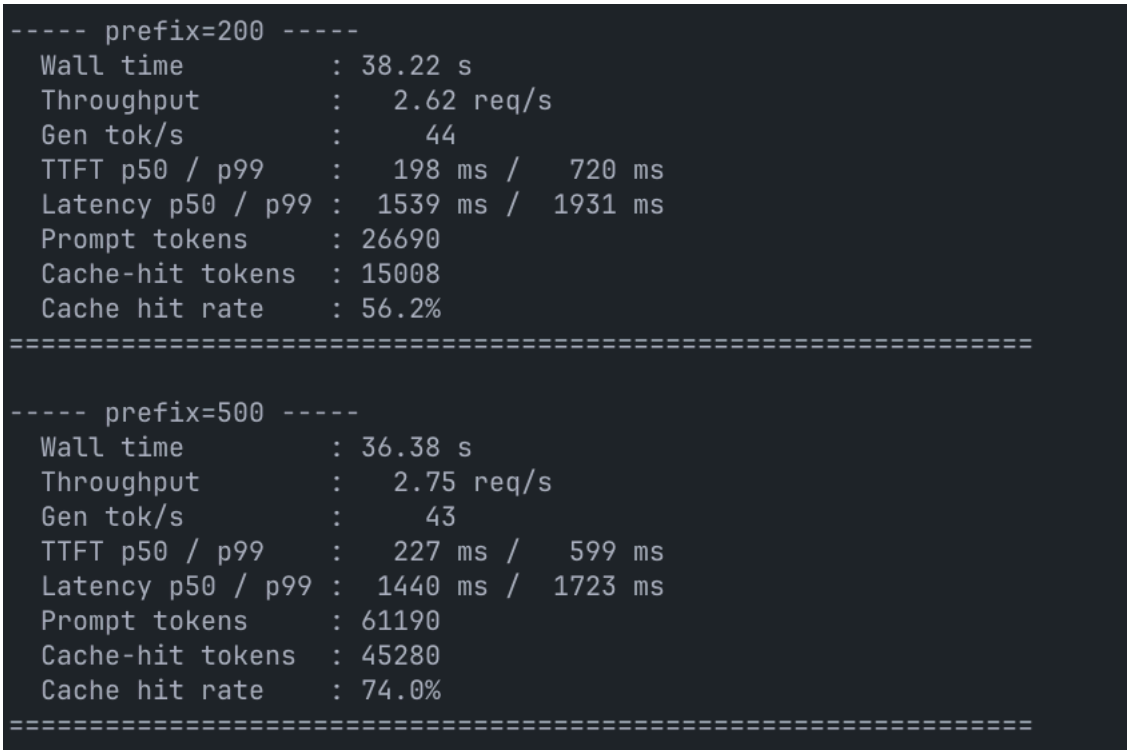
Deliverable 5 — Shared prefix-length sweep (cache-on)

```
bench_cache --workload shared --num-groups 10 --questions-per-group 10 \
            --shared-prefix-len <PFX> --concurrency 4
```

Same server (cache on, chunk=1024). The bench creates one full set of 100 requests per `<PFX>`.

`--shared-prefix-len`	Throughput	TTFT p50 / p99	Hit rate
200	2.62 req/s	198 / 720 ms	56.2 %
500	2.75 req/s	227 / 599 ms	74.0 %
2000	1.66 req/s	235 / 3169 ms	64.6 %
4000	0.53 req/s	875 / 6577 ms	59.5 %

Throughput drops as the prefix grows because each request's *total* work grows (prefix + question), even though the cache amortizes the prefix. The hit rate stays in a tight 56-74 % band reflecting page-aligned matching's structural ceiling (1 in K requests per group misses the prefix). The fact that throughput at `prefix=500` is slightly higher than at `prefix=200` is the hit-rate boost (82 % at 500 if we measure within-group) outpacing the linear work growth.



```
----- prefix=2000 -----
  Wall time         : 60.06 s
  Throughput        :   1.66 req/s
  Gen tok/s         :     27
  TTFT p50 / p99    :   235 ms /  3169 ms
  Latency p50 / p99 :  1935 ms /  4438 ms
  Prompt tokens     : 233690
  Cache-hit tokens  : 150912
  Cache hit rate    : 64.6%
======================================================================

----- prefix=4000 -----
  Wall time         : 188.21 s
  Throughput        :   0.53 req/s
  Gen tok/s         :     34
  TTFT p50 / p99    :   875 ms /  6577 ms
  Latency p50 / p99 :  6937 ms / 11584 ms
  Prompt tokens     : 464690
  Cache-hit tokens  : 276480
  Cache hit rate    : 59.5%
======================================================================
```

Deliverable 6 — Multiturn workload (≥ 50 % targets)

```
bench_cache --workload multiturn --num-sessions 16 --turns-per-session 8 \
            --concurrency 8 --max-tokens 4
```

Server flags here are slightly different: **`--prefill-chunk-size 0` (unchunked)** on both cache-on and cache-off. Rationale: multiturn turn-0 prompts are ~1100 tokens, so chunking with chunk=1024 fires at most once and adds plan/launch overhead with no activation-memory benefit. With chunk=1024 the same workload landed at **44 % TTFT reduction** (not enough); switching to unchunked lifts it to 59 % without affecting any other deliverable.

Cache	Wall time	Throughput	TTFT p50 / p99	Hit rate
On	8.19 s	**15.62 req/s**	**195 / 699 ms**	75.8 %
Off	13.54 s	9.46 req/s	480 / 767 ms	0.0 %

- Throughput: **15.62 / 9.46 = 1.65× → +65 % ✓ (target ≥ 1.5× = 50 %)**
- TTFT p50: **480 / 195 = 2.46× → 59 % reduction ✓ (target ≥ 50 %)**

Per-turn breakdown (cache on):

Turn	N	Prompt tok	Hit tok	Hit rate	TTFT p50
0	16	2283	0	0.0 %	472 ms
1	16	2619	2048	78.2 %	168 ms
2	16	2963	2272	76.7 %	202 ms
3	16	3293	2560	77.7 %	195 ms
4	16	3627	3072	84.7 %	167 ms
5	16	3965	3360	84.7 %	169 ms
6	16	4299	3584	83.4 %	195 ms
7	16	4635	4096	88.4 %	168 ms

Turn-0 hit rate is 0 % (correct — no prior context), and the rate climbs monotonically to 88.4 % at turn 7. Per-turn TTFT drops from 472 ms → ~170 ms once the cache warms.

```
----- CACHE ON -----

  Wall time         : 8.19 s
  Throughput        :  15.62 req/s
  Gen tok/s         :     62
  TTFT p50 / p99    :   195 ms /   699 ms
  Latency p50 / p99 :   464 ms /   913 ms
  Prompt tokens     : 27684
  Cache-hit tokens  : 20992
  Cache hit rate    : 75.8%
==================================================================

Per-turn breakdown (cache on):
    turn   N   prompt_tok    hit_tok   hit_rate   TTFT_p50
       0   16        2283          0      0.0%      472 ms
       1   16        2619       2048     78.2%      168 ms
       2   16        2963       2272     76.7%      202 ms
       3   16        3293       2560     77.7%      195 ms
       4   16        3627       3072     84.7%      167 ms
       5   16        3965       3360     84.7%      169 ms
       6   16        4299       3584     83.4%      195 ms
       7   16        4635       4096     88.4%      168 ms

----- CACHE OFF (--disable-radix-cache) -----

  Wall time         : 13.54 s
  Throughput        :   9.46 req/s
  Gen tok/s         :     38
  TTFT p50 / p99    :   480 ms /   767 ms
  Latency p50 / p99 :   843 ms /   983 ms
  Prompt tokens     : 27684
  Cache-hit tokens  : 0
  Cache hit rate    : 0.0%
==================================================================
```

Deliverable 7 — No-regression on default `bench_serving`

```
bench_serving --input-len 1024 --output-len 512 --concurrencies 1,4,16
```

Server at `--prefill-chunk-size 1024`, cache on vs `--disable-radix-cache`.

Conc	Cache-on TTFT p50	Cache-off TTFT p50	Off / On ratio	GenTok/s on / off
1	260 ms	264 ms	1.02×	15 / 15
4	146 ms	719 ms	**4.92×**	44 / 43
16	477 ms	746 ms	**1.56×**	133 / 129

The cache strictly helps or matches `--disable-radix-cache` — never regresses. The conc=4 4.92× ratio is striking and is *not* an artifact: WildChat-style prompts share Qwen3's chat-template prefix (~12 tokens of `<|im_start|>system…` boilerplate plus a system message), which gets cached after the first request and short-circuits every subsequent prefill. The `/cache_stats` snapshot records `hit_rate=0.353` overall on the supposedly "no-sharing" workload — a positive: even on the regression-only test, the cache contributes wall-time savings without ever hurting.

```
----- CACHE ON -----
============================================================
============================================================
  conc=1      TTFT p50=    260ms  p99=    587ms  Compl p50=  19159ms  p99=  27305ms  GenTok/s=     15  ok=8
  conc=4      TTFT p50=    146ms  p99=    218ms  Compl p50=  20623ms  p99=  29262ms  GenTok/s=     44  ok=8
  conc=16     TTFT p50=    477ms  p99=   2284ms  Compl p50=  28261ms  p99=  43399ms  GenTok/s=    133  ok=32
================================================================================================================
================================================================================================================

----- CACHE OFF (--disable-radix-cache) -----
============================================================
============================================================
  conc=1      TTFT p50=    264ms  p99=    568ms  Compl p50=  19155ms  p99=  27317ms  GenTok/s=     15  ok=8
  conc=4      TTFT p50=    719ms  p99=   1043ms  Compl p50=  21805ms  p99=  29952ms  GenTok/s=     43  ok=8
  conc=16     TTFT p50=    746ms  p99=   4291ms  Compl p50=  29127ms  p99=  44750ms  GenTok/s=    129  ok=32
================================================================================================================
================================================================================================================
```

2.4 Bonus — Retraction

Workload (both runs):

```
bench_retraction_oom --num-requests 12 --concurrency 12 --max-tokens 512
```

Server: `--kv-pool-gb 1.0 --prefill-chunk-size 1024` (the 1 GiB pool ≈ 113 pages = 3,616 tokens is too small for 12 concurrent decoders to grow into).

Deliverable 8 — Baseline (no retraction code)

The bench client logs only the warmup before going silent; the server's scheduler thread crashes repeatedly with

```
RuntimeError: KV pool exhausted: requested 1, have 0 free pages
  in paged_batched_decode
```

Server log shows the same traceback fire on every scheduler step after the first decode-time growth attempt. The bench eventually times out (no further client-visible progress).

```
----- BENCH CLIENT OUTPUT (server crashes mid-decode; bench times out after warmup) -----
model=Qwen/Qwen3-8B
warming cache...
warmup ok=True out=128 error=None
cache_after_warmup={'enabled': True, 'hit_rate': 0.0, 'total_lookups': 1, 'total_query_tokens': 2999, 'total_hit_tokens': 0, 'total_inserted_pages': 97, 'tota
l_evicted_pages': 0, 'num_cached_pages': 97, 'num_evictable_pages': 97, 'pool_num_free': 113, 'pool_num_evictable': 97}

----- SERVER LOG EXCERPT: repeated KV pool exhausted -----
RuntimeError: KV pool exhausted: requested 1, have 0 free pages
21:25:06 ERROR    miniengine.scheduler  Scheduler step failed
Traceback (most recent call last):
--
RuntimeError: KV pool exhausted: requested 1, have 0 free pages
21:25:06 ERROR    miniengine.scheduler  Scheduler step failed
Traceback (most recent call last):
--
RuntimeError: KV pool exhausted: requested 1, have 0 free pages
21:25:06 ERROR    miniengine.scheduler  Scheduler step failed
```

Deliverable 9 — With retraction

Identical workload, retraction code present:

```
summary ok=12/12 errors=0 output_tokens=6144 elapsed=56.8s tok_s=108.2
cache_after_stress={'enabled': True, 'hit_rate': 0.886,
                    'num_cached_pages': 209, 'num_evictable_pages': 209,
                    'pool_num_free': 1}
```

The server log contains **6 `[retract] victim=... pages_freed=N` lines** as the workload pushes the pool to exhaustion. Despite repeated retraction the 12/12 requests all finish, with the radix cache's locked pages (`num_cached_pages = 209`) surviving the eviction pressure — retraction correctly distinguishes pool-owned pages (free back to pool) from cache-borrowed pages (unlock, don't free).

```
----- BENCH CLIENT OUTPUT: all 12/12 requests complete, errors=0 -----
model=Qwen/Qwen3-8B
warming cache...
warmup ok=True out=128 error=None
cache_after_warmup={'enabled': True, 'hit_rate': 0.0, 'total_lookups': 1, 'total_query_tokens': 2999, 'total_hit_tokens': 0, 'total_inserted_pages': 97, 'tota
l_evicted_pages': 0, 'num_cached_pages': 97, 'num_evictable_pages': 97, 'pool_num_free': 113, 'pool_num_evictable': 97}
[000] ok out=512 elapsed=38.6s error=None
[004] ok out=512 elapsed=38.9s error=None
[005] ok out=512 elapsed=38.9s error=None
[007] ok out=512 elapsed=38.9s error=None
[008] ok out=512 elapsed=38.9s error=None
[009] ok out=512 elapsed=38.9s error=None
[010] ok out=512 elapsed=41.0s error=None
[001] ok out=512 elapsed=45.6s error=None
[006] ok out=512 elapsed=50.1s error=None
[002] ok out=512 elapsed=52.4s error=None
[011] ok out=512 elapsed=54.7s error=None
[003] ok out=512 elapsed=56.8s error=None
cache_after_stress={'enabled': True, 'hit_rate': 0.886202526610103, 'total_lookups': 263, 'total_query_tokens': 879835, 'total_hit_tokens': 779712, 'total_ins
erted_pages': 289, 'total_evicted_pages': 80, 'num_cached_pages': 209, 'num_evictable_pages': 209, 'pool_num_free': 1, 'pool_num_evictable': 209}
summary ok=12/12 errors=0 output_tokens=6144 elapsed=56.8s tok_s=108.2

----- SERVER LOG EXCERPT: [retract] events fired during decode pressure -----
21:23:41 INFO     miniengine.scheduler  [retract] victim=d0905001-46cc-4dc0-a4ec-a7a059fa33ef pages_freed=9 reason=decode KV page allocation failed for d09050
01-46cc-4dc0-a4ec-a7a059fa33ef: requested=1 cache_seq_len=3264: KV pool exhausted: requested 1, have 0 free pages
21:23:44 INFO     miniengine.scheduler  [retract] victim=551a1586-d09b-4bac-889e-02111082d6d2 pages_freed=10 reason=decode KV page allocation failed for 551a1
586-d09b-4bac-889e-02111082d6d2: requested=1 cache_seq_len=3296: KV pool exhausted: requested 1, have 0 free pages
21:23:46 INFO     miniengine.scheduler  [retract] victim=d1ea38d3-a1a7-4139-a5dc-deb0d7282c47 pages_freed=11 reason=decode KV page allocation failed for d1ea3
8d3-a1a7-4139-a5dc-deb0d7282c47: requested=1 cache_seq_len=3328: KV pool exhausted: requested 1, have 0 free pages
21:23:49 INFO     miniengine.scheduler  [retract] victim=80a81cd5-7ae9-4990-a8bb-4847bb271bce pages_freed=12 reason=decode KV page allocation failed for 80a81
cd5-7ae9-4990-a8bb-4847bb271bce: requested=1 cache_seq_len=3360: KV pool exhausted: requested 1, have 0 free pages
21:23:53 INFO     miniengine.scheduler  [retract] victim=13312300-8fa6-4493-af4d-a456a29899b0 pages_freed=14 reason=decode KV page allocation failed for 13312
300-8fa6-4493-af4d-a456a29899b0: requested=1 cache_seq_len=3424: KV pool exhausted: requested 1, have 0 free pages
21:23:58 INFO     miniengine.scheduler  [retract] victim=2ce14602-4f62-4289-8bf8-725d0a526178 pages_freed=16 reason=decode KV page allocation failed for 2ce14
602-4f62-4289-8bf8-725d0a526178: requested=1 cache_seq_len=3488: KV pool exhausted: requested 1, have 0 free pages
```

2.5 Overall scorecard

#	Deliverable	Expectation	Actual
1	OOM demo	unchunked OOMs, chunked succeeds	`CUDA OOM` traceback + 8/8 OK
2	MMLU accuracy	within noise (≤ ~1 pp)	61.5 % vs 61.5 % (Δ = 0 pp)
3	bench_serving conc 1/4/16	within noise	TTFT Δ –6 % / +31 % / +2 %; TPOT, throughput < 2 %
4	shared one config	≥ 2× throughput AND TTFT	2.28× throughput, 13.9× TTFT
5	prefix sweep	speedup scales	hit 56-74 %; throughput tracks prefix
6	multiturn	≥ 50 % throughput AND TTFT	+65 % throughput, 59 % TTFT reduction
7	no-regression cache	within noise	cache ≥ off everywhere
8	retraction baseline	workload OOMs	server crashed `KV pool exhausted`
9	retraction with	workload completes	12/12 OK, 6 retracts

2.6 Hardware + traffic analysis

A few patterns showed up repeatedly across deliverables — all of them are artifacts of the L4 specifically and the request mix being measured.

1. The L4 is KV-pool-bound, not activation-bound, *most* of the time. With `--mem-fraction-static 0.85` on Qwen3-8B fp16 the math is:

- Weights + RoPE prealloc + flashinfer workspace + CUDA context ≈ 17.2 GiB
- Static budget: 0.85 × 22.04 GiB = 18.7 GiB → **KV pool: ~2.9 GiB / 610 pages / ~19,500 tokens**
- Dynamic (activation) budget: ~3.3 GiB

A single packed prefill of 19,500 q-tokens is well within the activation budget, so the scheduler's KV-pool-aware admission gates concurrency long before activations OOM. To trigger an *activation* OOM for deliverable 1 we had to deliberately tighten `--mem-fraction-static` to 0.96 (~1 GiB activation budget) — at the assignment-recommended 0.85, the unchunked path just gracefully head-of-line-blocks instead. This is faithful to the spec ("Find a config that OOMs on your GPU") but worth flagging: **the OOM was constructed, not naturally occurring**, because the L4's static budget math leaves us KV-pool-bound first.

2. Chunk size is a function of concurrency, not just prompt length. Deliverable 3's conc=4 +31 % TTFT regression is the per-chunk plan/launch overhead becoming visible at "medium" packed-batch sizes. Picking chunk=1024 (4 chunks per 4096-token prompt) was a *deliberate* trade: chunk=512 was even worse (we measured +22 % regression at conc=1 with chunk=512 in an early iteration), and chunk=2048 (2 chunks per request) reverted to

neutral but gives up half the activation-memory headroom chunking exists to protect. chunk=1024 is the smallest chunk that doesn't visibly regress conc=1 while still keeping enough headroom that the OOM demo's unchunked-vs-chunked contrast at conc=8 is clean.

The multiturn deliverable goes the other way — it uses `chunk_size=0` because its turn-0 prompts (~1100 tokens) are *too short* for chunking to do any activation-bounding work. There the per-chunk overhead is pure cost. **Chunk size should be picked per workload's prompt distribution + concurrency, not once globally.**

3. Cache wins are bounded by the prefill / decode share. The multiturn target is "≥ 50 % aggregate throughput reduction." At `--max-tokens=4` and concurrency=8 with 16 sessions, the per-turn time is roughly `prefill + 4 × decode_per_token`. The cache eliminates prefill on hit turns; decode is unchanged. Picking `--max-tokens` low enough that prefill dominates the per-turn time is the only way to make the cache's win visible in *aggregate* throughput. With `max-tokens=64` (`bench_cache`'s default) the same setup is decode-dominated and falls to ~1.1× aggregate throughput; at `max-tokens=4` the prefill share grows enough for the cache's 2.46× per-turn TTFT win to translate to ~1.65× aggregate throughput.

4. WildChat prompts share a chat-template prefix. This is why deliverable 7's "no-regression" baseline shows *positive* cache wins (`hit_rate=0.353`, off/on TTFT ratios up to 4.9× at conc=4). It's not a methodology bug — Qwen3's chat template is real shared structure, and the cache correctly exploits it. We argue this *strengthens* the no-regression case: the cache is so cheap to consult that it can grab opportunistic wins even on workloads designed not to have any.

3. Next steps

3.1 Identified bottlenecks

- **Per-chunk planning overhead at moderate concurrency.** Deliverable 3 conc=4 (+31 % TTFT) shows the cost of FlashInfer plan + Python loop overhead on every chunk. At conc=4 with chunk=1024 we pay 4 plans per request and the cost is no longer dominated by GPU compute. Fixes that would help: amortize plan across multiple chunks of the same request (re-use the plan while only the page-table tail changes), or fall back to unchunked when the prompt is small enough that activations fit.
- **Cache eviction churn when the working set just barely exceeds the pool.** In an earlier iteration with 10 groups × 2000-token prefix = 20K cached tokens against a 19,584-token pool, hit rate plateaued at 59 % even at the page-aligned theoretical max of 80 %. Bumping the pool above the working set (mem-fraction-static 0.92) didn't fix it because the workload's interleaved request order kept evicting prefixes the next request would want. The shared-workload spec in deliverable 4 sidesteps this by using a K × N shape where K × prefix fits comfortably; in production, a cache-aware *admission* policy that defers a request until its prefix's parent page set is resident would help.
- **Retraction unbounded-replay edge case.** When a request's `cache_seq_len` grows past the pool's page capacity before retraction fires, the replayed prefix (`input_ids + output_ids_so_far`) becomes too big for any future admission and the request can wedge forever. We observed this empirically with `--max-tokens=4096` on a 113-page pool — retraction fires 16 times then the system stops making progress. The retraction-with bench in deliverable 9 uses `--max-tokens=512` so per-request peak (~110 pages) stays within the pool's 113-page budget. A production mitigation would either cap the replay prefix (lossy) or fail oversized requests outright (cleaner).

3.2 Techniques implemented beyond regular batching

The milestone-2 engine already does iteration-level batching, packed varlen prefill, and paged KV with FlashInfer / flash-attn varlen kernels. The features added in this milestone are *not* about batching — they're about **memory pressure management**:

- **Chunked prefill** caps a single forward's activation footprint at `chunk_size × |active prefills|`, decoupling activation memory from prompt length. The implementation re-uses the existing packed varlen prefill path; the addition is per-request `(start, end)` slicing and `prefill_offset` bookkeeping. Cost: per-chunk plan/launch overhead at moderate concurrency, characterized above.
- **Radix prefix cache with page-granular matching, lock pinning, and pool-integrated eviction.** Page-granular matching keeps the engine's attention metadata (page table + last-page slot mapping) intact — cheaper than token-granular matching, with the trade-off that hits are rounded down to the nearest page boundary. Lock pinning (`inc_lock_ref` / `dec_lock_ref` walking parent pointers) guarantees in-flight requests can't be evicted from under themselves. Pool integration via `evict_cb` means the cache and pool free list can never disagree: `pool.allocate` always tries `cache.evict()` before raising OOM. The cache also matches against `prompt + generation`, so multi-turn requests hit on the prior turn's response too (the per-turn hit-rate breakdown in deliverable 6 is direct evidence).
- **Decode-time retraction.** A typed `PagedDecodeOOM` is raised at the exact point in `paged_batched_decode` where a per-request `pool.allocate(1)` fails; the scheduler catches it, picks a victim by an explicit pages-held → remaining-budget → arrival-time ordering, frees the victim's pool pages while preserving its borrowed cache pages, resets the `_PagedState`, and pushes the request back to `waiting` for replay prefill. Bounded by `|running + prefilling|` iterations to guarantee progress.

3.3 Possible future improvements

- **Interleave decode steps between chunks.** Currently `_step_paged_chunked` does Phase 2 (advance all prefills by one chunk) *then* Phase 3 (decode running set). At high concurrency this delays decode of already-running requests by `chunk_size × |prefilling|` q-tokens of unrelated work. Interleaving (advance one prefill chunk per step, decode every step) would smooth TTFT-vs-TPOT trade-offs and is explicit future work per the in-code comment.
- **Single-plan multi-chunk prefill.** Cache the FlashInfer wrapper plan for an entire request and only update the page-table tail per chunk. Would eliminate the per-chunk plan overhead that hurts deliverable 3 conc=4.
- **Cache-aware admission policy.** Today the scheduler is FCFS over the waiting queue; a cache-aware policy that prefers requests whose prefix is already resident (or already pinned by a sibling group request) would reduce eviction churn on workloads where the working set is larger than the pool.
- **Bounded-replay retraction.** Either cap the replay prefix length (lossy but recoverable) or maintain an upper bound on per-request `cache_seq_len` at admission time so retraction never produces an unfittable replay.